

[Marcar como hecha](#)

11.2. Generación de páginas con servlets

El concepto básico de una aplicación web Java es un servlet, que es una clase Java que tiene métodos que pueden generar una respuesta al cliente en forma de HTML. Dado que es una clase Java normal, cuyos métodos se realizan en el servidor, puede realizar cualquier cosa que una clase pueda y puede, por ejemplo, leer datos en una base de datos. Por lo tanto, un servlet puede generar dinámicamente un documento HTML cuyo contenido depende de la situación. De hecho, es raro que, en la práctica, veas estos servlets ya que sueles desarrollar sitios web dinámicos como páginas JSP o Facelets, pero en ambos casos las páginas se traducidas a servlets, así que esta sección sobre qué es un servlet.

Primer Servlet

Si trabajamos por ejemplo con NetBeans como IDE, al seleccionar nuevo proyecto, dentro de las opciones java ant, tenemos web application. Si seguimos el proceso, nos pedirá un server con el que trabajar.

WildFly es un buen server. Tomcat, de Apache, es quizás más conocido. La instalación no requiere más que la descarga y el señalar en que directorio tenemos lo descargado. Tomcat 9 no es una mala versión, actualmente está la 10, con la que se suele trabajar las versiones de Jakarta EE, pero no están todas las compatibilidades todavía (no creo que tarden). Dentro de la carpeta Tomcat, en la subcarpeta conf, es posible modificar el fichero tomcat-users.xml, de manera que las dos últimas líneas incluyan un nombre y un pass de usuario, quedarían así;

```
<user password="root" roles="manager-script,admin, manager-gui" username="root"/>
</tomcat-users>
```

Esto, que solamente habría que hacerlo una vez, es similar a WildFly (aconsejable la versión 24).

Una vez que NetBeans tiene el servidor, ya puede utilizarse con proyectos.

Sin complicarnos mucho más, una vez que se ha creado el proyecto, con el nombre que elegimos, es posible darle al triángulo central verde de modo que ejecute la aplicación.

lanzará nuestro navegador, en `http://localhost:8080/NombreDeProyecto/`

como el proyecto por defecto crea una página index.html, veremos un "TODO write content" dentro del body del html puede cambiarse con el servidor en ejecución. Todo lo que corresponde a la parte estática del proyecto, es posible cambiarlo de este modo.

Por el momento sólo hay un único archivo llamado index.html, que es un documento HTML común, y puede ser percibido como la página de inicio de la aplicación

Como se ha mencionado, un servidor web es un programa que se está ejecutando constantemente, y tal programa es fuera del mundo conocido por el nombre del servidor (que en realidad es una dirección web) y un número de puerto.

Un servidor web puede ser configurado para usar un número de puerto específico y Tomcat usa por defecto el puerto 8080 igual que el antiguo GlassFish.

Actualmente, la aplicación no tiene servlets. Añadimos un servlet. Esto se hace pulsando con el botón derecho del ratón sobre <paquete por defecto> y seleccionando Nuevo ->Servlet. En la siguiente ventana se da nombre al servlet (la ventana muestra una advertencia que puedes ignorar en este lugar) y cuando haga clic en Siguiente se obtiene la ventana en donde permite poner en URL pattern(s) la url con la barra "/" donde queremos que atienda. No es necesario, pero tenemos esa opción. En mi ejemplo, dejaré el nombre por defecto "WebApplication1" y en la ruta y nombre del servlet llamaré Home y /Home

este será su código:

```

*
* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
* Click nbfs://nbhost/SystemFileSystem/Templates/JSP_Servlet/Servlet.java to edit this template
*/

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

/**
 *
 * @author enrique
 */
@WebServlet(name = "Home", urlPatterns = {"/Home"})
public class NewServlet extends HttpServlet {

    /**
     * Processes requests for both HTTP <code>GET</code> and <code>POST</code>
     * methods.
     *
     * @param request servlet request
     * @param response servlet response
     * @throws ServletException if a servlet-specific error occurs
     * @throws IOException if an I/O error occurs
     */
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            /* TODO output your page here. You may use following sample code. */
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<title>Servlet NewServlet</title>");
            out.println("</head>");
            out.println("<body>");
            out.println("<h1>Servlet NewServlet at " + request.getContextPath() + "</h1>");
            out.println("</body>");
            out.println("</html>");
        }
    }

    // <editor-fold defaultstate="collapsed" desc="HttpServlet methods. Click on the + sign on the left to edit the code.">
    /**
     * Handles the HTTP <code>GET</code> method.
     *
     * @param request servlet request
     * @param response servlet response
     * @throws ServletException if a servlet-specific error occurs
     * @throws IOException if an I/O error occurs
     */
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        processRequest(request, response);
    }

    /**
     * Handles the HTTP <code>POST</code> method.
     *
     * @param request servlet request
     * @param response servlet response
     * @throws ServletException if a servlet-specific error occurs
     * @throws IOException if an I/O error occurs

```

?

```

    */
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        processRequest(request, response);
    }

    /**
     * Returns a short description of the servlet.
     *
     * @return a String containing servlet description
     */
    @Override
    public String getServletInfo() {
        return "Short description";
    } // </editor-fold>
}

```

Como se ve, automáticamente se genera mucho código. Aparte de lo comentado

en principio atendemos a la anotación

@WebServlet (name = "Time", urlPatterns = {"/Time"})

Si escribimos en el index.html en el body, claro

```

<h1>Simple página HTML de toa la vida</h1>
<h2><a href="Home">¿Qué hora es?</a></h2>

```

Si ahora ejecutas la aplicación, se abre un navegador y hay un enlace ¿Qué hora es?, que lleva a otra dirección: /Home

Debes observar la barra de direcciones, que te indica que es el resultado del servlet lo que aparece.

A continuación me detendré un poco más en el código de la clase servlet. En primer lugar, observa que la clase hereda de HttpServlet, que es la clase base de un servlet. Si consideras el método processRequest(), está claro que crea un documento HTML. El método tiene dos parámetros, que representan respectivamente una solicitud (de un navegador) y una respuesta. La comunicación entre el cliente y el servidor se produce según el protocolo HTTP, que es un protocolo simple basado en texto. Cuando el navegador ejecuta una petición, puede hacerlo de dos formas, que se denominan GET y POST, y la principal diferencia es cómo el navegador envía los datos junto con la petición. Cuando el navegador envía una petición a un servlet, se ejecuta uno de los métodos doGet() y doPost(), y ambos métodos tienen dos parámetros, que son referencias a objetos que representan respectivamente una petición y una respuesta. En este caso, no ocurre nada más que el envío de estos objetos al método processRequest(). Este inicializa el objeto respuesta a un documento HTML, tras lo cual determina una referencia al escritor del objeto. La referencia se utiliza para imprimir texto para el documento, texto que en este caso constituye un documento HTML. Cuando el bloque try sale del ámbito, la referencia se borra, lo que significa que el documento se termina y se envía una respuesta al cliente.

Ahora cambiaré el método processRequest() de la siguiente manera las primeras (la primera cambia) líneas dan pista de dónde:

```

out.println("<h1>El día y hora es: " + getTime() + "</h1>");
out.println("</body>");

out.println("</html>");
    }
}

```

```
private String getTime() {
    Calendar date = Calendar.getInstance ();
    return String.format("%d. %s %d %02d:%02d:%02d", date.get(Calendar.DATE),
        getMonth(date.get(Calendar.MONTH)), date.get(Calendar.YEAR),
        date.get(Calendar.HOUR), date.get(Calendar.MINUTE), date.get(Calendar.SECOND));
}
private String getMonth(int n){
    switch(n){
        case 0: return "Enero";
        case 1: return "Febrero";
        case 2: return "Marzo";
        case 3: return "Abril";
        case 4: return "Mayo";
        case 5: return "Junio";
        case 6: return "Julio";
        case 7: return "Agosto";
        case 8: return "Septiembre";
        case 9: return "Octubre";
        case 10: return "Noviembre";
        case 11: return "Diciembre";
    }
    return "";
}
```

Hay que importar la clase "Calendar" de Java.util.

Al ejecutarlo nos dice El día y la Hora es: fecha y hora

De hecho, sólo se ha cambiado una sentencia, (del HTML que se crea arriba línea a línea) pero ahora llama al método getTime(), que retorna una cadena con la fecha y hora actuales. El objetivo de todo es mostrar que un servlet puede contener todo el código Java que sea necesario, ya que un servlet es simplemente una clase Java. Otro objetivo es que mi servlet es ahora dinámico de modo que el html enviado al navegador depende de cuando se está ejecutando la aplicación. A continuación se muestra un ejemplo después de hacer clic en el enlace en la página de inicio.

No es necesario abrir la página haciendo clic en el enlace de la página de inicio. página de inicio, sino que también puede abrir directamente la página -y obtener el servlet que crea la página - introduciendo la dirección directamente en el navegador. Una vez que hayas escrito una aplicación web como la anterior, debe estar alojada en el servidor de aplicaciones antes de poder abrirla en el navegador. La tarea se llama despliegue, y era antes y todavía puede ser complejo, pero si la aplicación se desarrolla utilizando NetBeans, es todo sobre sí mismo como parte del proceso de construcción sin necesidad de hacer nada. Si examinas el proyecto, verás que bajo la carpeta dist hay un fichero con (en este caso) el nombre NewServlet.war y es un paquete que contiene los ficheros que requiere el proceso de despliegue. Si abre el navegador e introduce la dirección se abre la página de inicio de la aplicación. Esto corresponde al envío de una petición al el servidor local pidiéndole que ejecute la aplicación NewServlet y el servidor ha enviado como respuesta las páginas de inicio de la aplicación de la aplicación, que son index.html.

Última modificación: lunes, 13 de marzo de 2023, 11:16